Reinforcement Learning and Dynamic Optimization

Programming Assignment 1 - Stochastic Bandits

Horn Pavlos – 2022030203

➢ Environment Setup

System Parameters:

Random Seed	Number of Channels (K)	Number of Users	SNR range	Noise level (e)	Monte Carlo samples per arm pull
3	5	2	[0.8, 2.0]	0.6	100

Using **np.random.uniform(min_snr, max_snr, size=(2,K))**, we generate the clean SNR for each user for each channel.

SNR's per Channel					
	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5
User 1	1.46095748	1.64977739	1.14908569	1.41299313	1.87153635
User 2	1.87555171	0.95070237	1.04869145	0.86176064	1.32897181

➢ Case A: Joint Feedback

- ➢ In this case the algorithm only sees r(t) , which is the sum of the two rates:

$r(t)=R_1(i)+R_2(j)$, where $R_1(i),R_2(j)=\log(1+10s)$ and $s \sim Unif[s_{i,j}-\varepsilon, s_{i,j}+\varepsilon]$ and $\varepsilon=0.6$

- ➢ That is why, for this case a "arm" is the combination of two channels, where each channels must be different from the other. Therefore, we had in this case 20 arms (Total arms: 4*5=20).
- ➢ Also in Case A, the objective of our algorithm is to learn the true expected reward so that it can know which "arm" is the optimal.

UCB Formula used:

$$\mathbf{UCB_i(t)= \hat{\mu}_i(t) +2 * (Max\ Rate - Min\ Rate)\sqrt{\frac{2\log(T)}{N_i(t)}}},$$

- $\hat{\mu}_i(t)$ is the mean reward for arm a after t rounds.
- $2 * (\text{Max Rate} - \text{Min Rate})\sqrt{\frac{2\log(T)}{N_i(t)}}$ is the UCB bonus term which can be broken down to two parts:
  - ✓ $2 * (\text{Max Rate} - \text{Min Rate})$, is the range of possible rewards and *2 due to the fact we have two users. This was the tough part to come up with, and I had some assistance from Ai agents, since in class the examples done where for ucb which had range [0,1], therefore no σ was needed since.
  - ✓ $\sqrt{\frac{2\log(T)}{N_i(t)}}$, which is based on the lecture notes and is the part that ensures each arm is tried sufficiently often.

In the following table are the theoretical mean reward, the practical mean reward and the Δi for both the experiments with the different values of T:

Arm Index	Pair	Theoretical Mean Reward	Δi(μ*-μi)	Practical Mean Reward(T=2000)	Practical Mean Reward(T=2000)
0	(0,1)	2.21487079	0.3756224	2.17692788	2.17666361
1	(0,2)	2.25359446	0.33689872	2.21916188	2.22203239
2	(0,3)	2.17645808	0.41403511	2.13412371	2.13175766
3	(0,4)	2.34841474	0.24207845	2.32406954	2.32363688
4	(1,0)	2.5386712	0.05182199	2.5230404	2.52303204
5	(1,2)	2.30318619	0.287307	2.27321944	2.27178781
6	(1,3)	2.2260498	0.36444339	2.18228282	2.18411598
7	(1,4)	2.39800646	0.19248673	2.37513568	2.37646809
8	(2,0)	2.39228063	0.19821256	2.3675763	2.36796678
9	(2,1)	2.11807194	0.47242124	2.07832673	2.07281394
10	(2,3)	2.07965923	0.51083396	2.03302908	2.03340432
11	(2,4)	2.25161589	0.33887729	2.21582509	2.22010463
12	(3,0)	2.47552536	0.11496783	2.4568139	2.45702096
13	(3,1)	2.20131667	0.38917652	2.16832354	2.16369708
14	(3,2)	2.24004034	0.35045284	2.20543308	2.20611254
15	(3,4)	2.33486062	0.25563257	2.31044038	2.30911224
16	(4,0)	2.59049319	0 (Optimal)	2.57786624	2.57680966
17	(4,1)	2.3162845	0.27420869	2.28083901	2.28303889
18	(4,2)	2.35500818	0.23548501	2.32313658	2.32600052
19	(4,3)	2.27787179	0.3126214	2.23705511	2.23770588

- To justify that our algorithm works properly I also calculated the regret bound. For the calculation, I used the following formula which can be found from the lecture notes with a small addition from an Ai agent:

$$\mathbf{E[R(T)]=\sigma^2(P(Good)*\sum_{i=0}^{K} N_i(t)\Delta_i + P(Bad) * \sum_{i=0}^{K} N_i(t) * \Delta_i)},$$

$\sigma^2 = (\frac{\text{Max Rate} - \text{Min Rate}}{2})^2$ is the sub-Gaussian parameter since our reward is bounded in [a,b], it helps introduce the noise levels.

Which can be simplified:

$P(Bad) * \sum_{i=0}^{K} N_i(t) * \Delta_i < KT^{-3} * T * 1$ which is close to 0 so we can ignore,

$$P(Good) * \sum_{i=0}^{K} N_i(t) * \Delta_i \leq \sum_{i=0}^{K} \frac{8logT}{\Delta_i}$$

Therefore, the actual formula used was:

$$\mathbf{E[R(T)] \leq \sigma^2 \sum_{i=0}^{K} \frac{8logT}{\Delta_i}}$$

Case B	
	$Bound$
T=2000	2022.060068226934
T=20000	2634.6143530662057

Case B: Per-User Feedback

- In this case the algorithm sees both individual rates separately.

- That is why, for this case we have two independent sets of "arms". One arm for User 1 and one arm for User 2, with 5 individual channels each.
- Also in Case B, the objective of our algorithm is to learn the true expected reward of the combination of the "arms" so that it can know which "arm" is the optimal. More specifically, the algorithm maintains separate estimates for both users and then tries to find the best pair.

UCB Formula used:

For each user i ∈ {0,1}:

$$\mathbf{UCB_i(t)} = \hat{\mu}_i(t) + \sigma\sqrt{\frac{2\log(T)}{N_i(t)}},$$

- $\hat{\mu}_i(t)$ is the mean reward for arm a after t rounds.
- $\sigma\sqrt{\frac{2\log(T)}{N_i(t)}}$ is the UCB bonus term which can be broken down to two parts:
  - ✓ σ =Max Rate − Min Rate , which is range of possible rewards. This was the tough part to come up with since in class the examples done where for ucb which had range [0,1], therefore no σ was needed since.
  - ✓ $\sqrt{\frac{2\log(T)}{N_i(t)}}$, which is based on the lecture notes and is the part that ensures each arm is tried sufficiently often.

❗ It is important to note that if the same channels had the biggest ucb then the code looks for the next best combination using two different channels by going to the second best of the two users and checking which is larger.

In the following tables are the theoretical mean reward, the practical mean reward and the Δi for both the experiments with the different values of T, for both users:

User 1				
	Theoretical Mean Reward	Δi(μ*-μi)	Practical Mean Reward(T=2000)	Practical Mean Reward(T=2000)
Channel 0	1.19339107	0.10141371	1.1815701	1.18222622
Channel 1	1.2429828	0.05182199	1.23417552	1.23417808
Channel 2	1.09659223	0.19821256	1.07818519	1.07780881
Channel 3	1.17983695	0.11496783	1.16839669	1.16804735
Channel 4	1.29480479	0 (Optimal)	1.28777329	1.28791748

User 2				
	Theoretical Mean Reward	Δi(μ*-μi)	Practical Mean Reward(T=2000)	Practical Mean Reward(T=2000)
Channel 0	1.2956884	0 (Optimal)	1.28872346	1.28878192
Channel 1	1.02147971	0.27420869	0.99403336	0.99289592
Channel 2	1.06020339	0.23548501	1.03850702	1.03958554
Channel 3	0.983067	0.3126214	0.94788363	0.94892309
Channel 4	1.15502366	0.14066474	1.14104707	1.14074518

- To justify that our algorithm works properly I calculated the regret bound. For the calculation, I used the previously used formula with some alterations:

Initially since this time, we objectively do a ucb twice. One time for each user.

Therefore, since the bound was computed using the following formula:

$$\mathbf{E[R(T)]} \leq \sigma^2 \sum_{i=0}^{K} \frac{8logT}{\Delta_i}$$

This time we would have :

$$\mathbf{E[R(T)]} \leq \sigma^2 \sum_{i=0}^{K} \frac{8logT}{\Delta_{i,user1}} + \sigma^2 \sum_{i=0}^{K} \frac{8logT}{\Delta_{i,user2}}$$

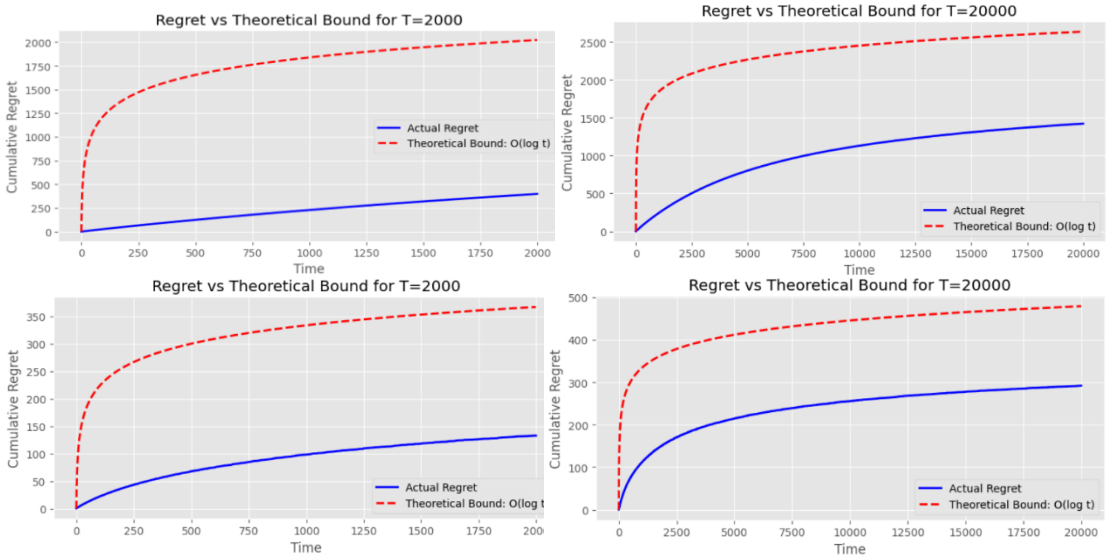
Case B	
	Bound
T=2000	367.3258672721232
T=20000	478.6020046457971

Comparison and Results:

- Below we can notice the plots that demonstrate that our algorithm works for both ranges of T and both cases, since we notice the regret is sublinear which indicates that it is learning. However, we notice that the longer the horizon, the algorithm achieves lower average regret per round, demonstrating improved performance with more time.

- **In the code there are more plots that indicate the algorithm learns however for size issues the above have been chosen.**

- Furthermore, the following plots prove that our Bound, in both cases and for both T, is set correctly as our cumulative regret is well below it.



- Prior to implementing the two cases it was expected that Case B would learn faster since it has more feedback from the user, which allows it to decompose a 20-arm problem into two 5-arm problem. This is also reflected in the two following graphs.